

§ 2. Вычислимые функции

Начнем с двух замечаний об алгоритмах, а точнее, о машинах Тьюринга, которые мы начали изучать на прошлой лекции.

Замечание 1. У машин Тьюринга имеются «области определения», аналогичные известному вам из матем. анализа понятию ООФ.

Замечание 2. Было сказано (вскользь), что теоретически для описания всевозможных алгоритмов может потребоваться «большой» алфавит, вплоть до бесконечного. Например, ситуацию $\mathcal{A} = \mathbb{N}$ можно представлять как вполне реальную в «каких-то» конкретных задачах. А в наших конкретных обсуждениях мы пока использовали лишь двоичный алфавит $\mathcal{A} = E = \{0, 1\}$. Можно ли «сгладить» такое огромное различие?

Предложение 1. Множество слов в не более чем счетном алфавите $\mathcal{A}$ можно однозначно закодировать в алфавите $E = \{0, 1\}$.

Док-во. Закодируем натуральное число n (или букву a_n) набором 01...10 из n единиц, обрамляя их отдельными нулями с двух сторон. Ч.Т.Д.

Вопрос: а как закодировать $0 \in \mathbb{N}$? Придется слегка модифицировать идею. Набор из нескольких натуральных чисел (слово $A = a_1 a_2 \dots a_n$ в алфавите $\mathbb{N}$) закодируем чуть сложнее:

$$A \rightarrow 1^{a_1+1} 0 1^{a_2+1} 0 \dots 0 1^{a_n+1}. \quad (1)$$

Другими словами, каждая буква (натуральное число n) будет описываться набором из $(n + 1)$ единицы, а разделять буквы удобно нулями. Тогда

$$0 \rightarrow 1, 1 \rightarrow 2, \dots, n \rightarrow (n + 1), \dots \quad (2)$$

Схему кодирования (2) будет удобно использовать при обсуждении машин Тьюринга, а вообще работают с теми алфавитами и их кодировками, которые более удобны в конкретных задачах.

Теперь мы переходим к понятию функций (от нескольких числовых переменных), приближенному к машинам Тьюринга.

Определение 1. Будем называть *частичной числовой функцией* любое отображение

$$f : \mathbb{N} \times \dots \times \mathbb{N} \rightarrow \mathbb{N}, \quad (3)$$

определенное на **некоторых** наборах $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}^n$.

Обозначение $f(x_1, \dots, x_n)$ такой функции является стандартным и похожим, например, на обозначение булевских функций. Но только теперь $x_1, \dots, x_n$ — это не булевские значения, а **натуральные числа**.

Определение 2. Частичная числовая функция $f(x_1, \dots, x_n)$ называется *вычислимой*, если существует машина Тьюринга T_f , **вычисляющая** данную функцию, т.е.:

1) если $f(\alpha)$ определено, то машина T_f применима к слову α , и результатом ее работы с этим словом является натуральное число $f(\alpha)$ (записанное на ленте);

2) если $f(\alpha)$ не определено, то либо T_f не применима к слову $\alpha = (\alpha_1, \dots, \alpha_n)$, либо применима, но результат ее применения к набору чисел α НЕ является кодом НИКАКОГО натурального числа.

Замечание. Обычно интересуются применимостью МТ только к «нужным» словам, а разными «неинтересными» данными на ленте «голову не забивают».

Пример 1. $f(x) = 0$ функция одного натурального переменного. Вычислима ли она ?

$$\Pi : \begin{array}{l} q_1 1 q_1 0 R \\ q_1 0 q_0 1 S \end{array}$$

Эта программа вычисляет нулевую функцию, значит, нулевая функция **вычислима**.

Пример 2. $f(x_1, x_2) = x_1 + x_2$ — тоже **вычисляемая** функция.

Описание задачи «на ленте»:

$$\begin{array}{ccccc} 1...1 & 0 & 1...1 & \rightarrow & 1...1 \\ (x_1 + 1) & & (x_2 + 1) & & (x_1 + x_2 + 1) \end{array}$$

Здесь напрашиваются очевидные команды

$$\Pi : \begin{array}{l} q_1 1 q_1 1 R \\ q_1 0 q_1 1 R \end{array},$$

которые, конечно же, нужно более аккуратно выстраивать.

И поэтому предлагается сначала на этом примере разделить саму задачу на микро-задачи, а потом уже заполнить командами отдельные микро-программы и сшить целую программу. Итак,

1) пробег по первому слагаемому слева направо;

2) замена разделяющего нуля на единицу.

А дальше возможны (достаточно очевидные) варианты: либо продолжать движение вправо, либо вернуться налево к началу слова. Выберем второй вариант:

3) пробег по первому слагаемому справа налево

4) уничтожение «лишних» единиц в начале итогового слова :

$$(x_1 + 1) + 1 + (x_2 + 1) = (x_1 + x_2 + 3).$$

А нужно оставить на ленте $(x_1 + x_2 + 1)$ единичек, т.е. две — лишние.

Итоговая программа может выглядеть, например, вот так:

$$\begin{array}{lll} \Pi_1 : & q_1 1 q_1 1 R & \Pi_2 : q_2 1 q_2 1 L \\ & q_1 0 q_2 1 S & q_2 0 q_3 0 R \end{array} \quad \Pi_3 : \begin{array}{l} q_3 1 q_4 0 R \\ q_4 1 q_5 0 R \\ q_5 1 q_0 1 S \end{array}$$

Замечание 1. Две последних команды в этой программе можно заменить одной $q_4 1 q_0 0 R$ (если уже есть минимальный опыт и понимание программ для МТ).

Замечание 2. Функция $S(x) = x + 1$ — функция следования, очевидно, тоже **вычислима**.

Пример 3. Функция выбора $I_{n,m}(x_1, \dots, x_n) = x_m$ ($1 \leq m \leq n$), выделяющая (выбирающая) из упорядоченного набора n натуральных чисел одно число с номером m .

Утверждение. Для любых **фиксированных** n, m ($1 \leq m \leq n$) функция выбора **вычислима**.

Замечание. Момент фиксации n, m является очень важным. Не следует думать, что существует ОДНА программа, вычисляющая любую из БЕСКОНЕЧНОГО КОЛИЧЕСТВА функций выбора ! Для каждой пары (n, m) имеется СВОЯ такая программа.

Для иллюстрации рассмотрим функцию $I_{3,2}(x_1, x_2, x_3)$ и обсудим ее возможное блочное устройство.

1) пошаговое «стирание» первого аргумента x_1 из набора x_1, x_2, x_3 :

$$q_1 1 q_1 0 R;$$

2) переход через ноль, отделяющий x_1 от x_2 ; пробег по второму аргументу из трех:

$$q_1 0 q_2 0 R; \quad q_2 1 q_2 1 R;$$

3) переход через ноль, отделяющий x_2 от x_3 ; пошаговое уничтожение третьего аргумента x_3 ; возвращение к концу слова x_2 — **Упражнение.**

Рекурсивные функции и операции C , Пр , μ

Определение 2. Пусть заданы частичные числовые функции

$$\begin{aligned} &f(y_1, \dots, y_m), \\ &g_1(x_1, \dots, x_n), \\ &\dots\dots\dots \\ &g_m(x_1, \dots, x_n). \end{aligned} \tag{4}$$

Тогда *суперпозиция* (C) функций $f, g_1, \dots, g_n$ — это новая частичная числовая функция

$$F(x_1, \dots, x_n) = f(g_1(x_1, \dots, x_n), \dots, g_m(x_1, \dots, x_n)). \tag{5}$$

Такая запись является вполне стандартной в математике, но все же поясним ее. Если на наборе натуральных чисел $\alpha = (a_1, \dots, a_n)$ определены ВСЕ внутренние функции $g_1, \dots, g_n$, т. что

$$\begin{aligned} &g_1(a_1, \dots, a_n) = b_1, \\ &\dots\dots\dots \\ &g_m(a_1, \dots, a_n) = b_m \end{aligned}$$

и внешняя функция f определена на наборе $\beta = (b_1, \dots, b_m)$, то суперпозиция F также считается определенной на наборе $\alpha = (a_1, \dots, a_n)$ и $F(\alpha) = f(\beta) = f(b_1, \dots, b_m)$.

А если на наборе $\alpha = (a_1, \dots, a_n)$ не определена хотя бы одна из внутренних функций $g_1, \dots, g_n$

или

определены все внутренние функции $g_1, \dots, g_m$, но на наборе $\beta = (b_1, \dots, b_m)$ НЕ определена внешняя функция $f(y_1, \dots, y_m)$, то и вся сложная функция F НЕ ОПРЕДЕЛЕНА на наборе α .

Замечание. Если функции $f, g_1, \dots, g_m$ — всюду определенные, то их суперпозиция C также всюду определена.

Пример. $f(y_1, y_2) = y_1 \cdot y_2$, $g_1(x_1, x_2) = x_1 + x_2$, $g_2(x_1, x_2) = x_1 - x_2$. Тогда

$$F(g_1(x_1, x_2), g_2(x_1, x_2)) = (x_1 + x_2) \cdot (x_1 - x_2) = x_1^2 - x_2^2.$$

Определение 3. Пусть заданы две частичные числовые функции

$$f(x_1, \dots, x_{n-1}), \\ g(x_1, \dots, x_{n-1}, x_n, x_{n+1}).$$

Примитивной рекурсией этих функций ($\text{Пр}(f, g)$) называется новая функция $h(x_1, \dots, x_n)$, определяемая *индукцией* по переменной x_n :

- 1) $h(x_1, \dots, x_{n-1}, 0) = f(x_1, \dots, x_{n-1})$,
- 2) $h(x_1, \dots, x_{n-1}, y + 1) = g(x_1, \dots, x_{n-1}, y, h(x_1, \dots, x_{n-1}, y))$, при $y \geq 0$.

Замечание. Рассматривая операцию "Пр нужно четко выделять «основного массива» переменных ($x_1, \dots, x_{n-1}$ и позиции двух «особых» переменных рекурсии. Одна из этих переменных — «счетчик», по которому осуществляется индукция, а другая — это значения результирующей функции. Каждое очередное значение этой переменной вычисляется с использованием предыдущего значения (в этом и есть смысл термина «рекурсия»).

Пример. $f(x) = x$, $g(x_1, x_2, x_3) = x_1 + x_2$. $\text{Пр}(f, g) = h(x_1, x_2) = ?$

Здесь «основной массив» содержит единственную переменную $x_1 = x$, счетчиком считаем переменную x_2 функции $g(x_1, x_2, x_3) = x_1 + x_2$, а третья, фиктивная (!), переменная x_3 этой функции выделяется для получения значений итоговой рекурсии.

Итак,

$$\begin{aligned} h(x_1, 0) &= f(x_1) = x_1, \\ h(x_1, 1) &= h(x_1, 0 + 1) = g(x_1, 0, h(x_1, 0)) = x_1 + 0 = x_1, \\ h(x_1, 2) &= h(x_1, 1 + 1) = g(x_1, 1, h(x_1, 1)) = x_1 + 1, \\ &\dots\dots\dots \\ h(x_1, n + 1) &= g(x_1, n, h(x_1, n)) = x_1 + n, \\ &\dots\dots\dots \end{aligned}$$

$$\text{Вывод: } h(x, y) = \begin{cases} x, & y = 0, \\ x + y - 1, & y \geq 1. \end{cases}$$

Пример-упражнение. $f(x_1, x_2) = (x_1 + x_2)$ — это примитивная рекурсия $I_{1,1}(x) = x$ и $S(x)$.

Замечание 1. Умножение двух натуральных чисел - это «многократно повторенное сложение», т.е. рекурсия сложения $x + y$ с «чем-то еще». Полезно подумать о «чем-то еще» !

Замечание 2. Определение операции $\text{Pr}(f, g)$ выглядит вполне понятным образом. Но мы ведь вводим эту операцию для **частичных** числовых функций. Что если функция

$$g(x_1, \dots, x_{n-1}, y, h(x_1, \dots, x_{n-1}, y))$$

или даже $f(x_1, \dots, x_{n-1})$ где-то не определены ?

Тогда и $h(x_1, \dots, x_{n-1}, y + 1)$ на «плохом» наборе переменных тоже не определена, а на последующих значениях счетчика y тем более !

Операция минимизации

Определение 4. Пусть $f(x_1, \dots, x_n)$ — частичная числовая функция. *Минимизация* этой функции по переменной x_n — это новая частичная числовая функция $g(x_1, \dots, x_n) = \mu_{x_n}(f) = \mu_y(f) = \mu(f)$, которая строится по многоступенчатому рецепту:

1) для каждого набора переменных $\alpha = (a_1, \dots, a_n)$ рассматривается уравнение

$$f(a_1, \dots, a_{n-1}, y) = a_n; \quad (6)$$

2) у этого уравнения обсуждаются решения (по переменной y). Если решений нет, то функция $\mu(f)$ НЕ ОПРЕДЕЛЕНА на наборе чисел $\alpha = (a_1, \dots, a_n)$;

3) если уравнение (6) имеет решения, то нужно выбрать минимальное из таких решений (оно всегда находится единственным образом !):

$$y_0 = \min\{y \in \mathbb{N} \mid f(a_1, \dots, a_{n-1}, y) = a_n\}; \quad (7)$$

4) если при этом определены еще и

$$f(a_1, \dots, a_{n-1}, 0), \dots, f(a_1, \dots, a_{n-1}, y_0 - 1),$$

то полагаем

$$(\mu_y(f))(a_1, \dots, a_{n-1}, a_n) = g(a_1, \dots, a_{n-1}, a_n) = y_0; \quad (8)$$

5) если даже нашлось y_0 по формуле (7), НО хотя бы при одном натуральном $i < y_0$ НЕ ОПРЕДЕЛЕНО значение $f(a_1, \dots, a_{n-1}, i)$, то считается , что $\mu_y(f) = g(x_1, \dots, x_{n-1}, x_n)$ также НЕ ОПРЕДЕЛЕНА на наборе $\alpha = (a_1, \dots, a_n)$.

Замечание 1. Операцию минимизации можно прокомментировать как решение (механизированное, алгоритмическое) **неявного** уравнения (6). А как компьютер (ИИ) на современном этапе развития ИТ может решать такие уравнения ? Только перебором ! Это объясняет почему нужно при («случайно») найденном решении y_0 такого уравнения рассматривать значения функции f на меньших, чем y_0 , значениях последнего аргумента.

Пример. $f(x_1, x_2) = x_1 + x_2$. Построим минимизацию этой функции по второй переменной. Это означает необходимость изучения уравнения

$$x_1 + y = x_2 \quad (9)$$

Здесь $g(x_1, x_2) = \mu_y(f(x_1, y) = y_2) = \mu_y(x_1 + y = x_2)$.

Что можно сказать о решениях уравнения (9) относительно переменной y ? Когда они существуют, а когда — нет ?

Школьный ответ: решение уравнения (9) всегда (при любых x_1, x_2) существует, единственно и описывается формулой $y = x_2 - x_1$.

Что нас может не устроить в этом школьном ответе ? Только то, что мы сейчас работаем на множестве $\mathbb{N}$ натуральных чисел. А потому школьный ответ придется скорректировать:

$$\mu_y(f) = g(x_1, x_2) = \begin{cases} x_2 - x_1, & x_2 \geq x_1, \\ \emptyset, & x_2 < x_1. \end{cases}$$

Замечание 1. Здесь символ $\emptyset$ можно проговорить словами: «не определено».

Замечание 2. Минимизация в этом примере привела к получению из всюду определенной начальной функции «настоящей» частичной функции. Для двух предыдущих операций С, Пр такое было невозможно.

Замечание 3. Если начальная функция является **всюду определенной**, то при построении ее минимизации (по любому аргументу) итоговая **проверка** п. 5 из определения НЕ требуется. А вообще говоря, без этого пункта определение 4 не является полным.

Еще **Пример.** Минимизировать **по переменной** x_1 частичную числовую функцию

$$f(x_1, x_2) = \begin{cases} x_2 + 2, & x_2 \geq x_1, \\ \emptyset, & x_2 < x_1. \end{cases}$$

Начало решения: Обозначим искомую функцию через $g(x_1, x_2)$ и попытаемся ее понять. Для этого рассмотрим уравнение

$$f(y, x_2) = x_1.$$

Как его решать (или, точнее, исследовать) ? Для начинающих есть подсказка: по точкам.

1) $P_1(0, 0) \in \mathbb{N}^2$. Тогда для этой точки возникает уравнение

$$f(y, 0) = 0.$$

Есть ли у него решения ? Из двух строк, описывающих функцию f , нам здесь может быть полезна только первая строка. Тогда надо разобраться с уравнением

$$0 + 2 = 0.$$

Ясно, что это уравнение **ни при каких** y не имеет решения. Следовательно, $g(0, 0)$ НЕ определена !

Далее можно перебрать еще несколько точек и в процессе перебора попытаться понять более общие свойства функции $g(x_1, x_2)$. На каком-то этапе таких движений (у студентов, претендующих хотя бы на четверку) должно появиться ПОЛНОЕ внятное описание искомой функции.

Контрольный вопрос: есть ли (у кого-нибудь) понятная **целочисленная** точка из первого квадрата плоскости, в которой искомая минимизация была бы определена ?

Подсказка: функция $g(x, y)$ является **нигде не определенной** частичной функцией.

После таких примеров вернемся к более общим обсуждениям.

Классы рекурсивных функций

Определение 5. Класс *основных* функций

$$A = \{0(x), S(x), I_{n,m}(x_1, \dots, x_n), 1 \leq m \leq n, n = 1, 2, \dots\};$$

класс P_{pr} *примитивно-рекурсивных* функций, т.е функций, получаемых из основного класса при помощи операций С, Пр;

класс P_r *рекурсивных* функций, т.е **всюду определенных** функций, получаемых из основного класса при помощи всех трех введенных выше операций С, Пр, μ ;

класс P_{cr} *частично-рекурсивных* функций, т.е. функций, получаемых из основного класса при помощи всех трех операций S , Pr , μ (без каких-либо ограничений);

класс P_{comp} *вычислимых* функций, т.е., функций, для которых имеются вычисляющие их алгоритмы.

Возникают естественные вопросы о связях этих классов. И есть несколько очевидных ответов:

$$1) A \subset P_{pr} \subset P_r \subset P_{cr}, \quad (10)$$

$$2) A \subset P_{comp}.$$

Замечание - упражнение.

$$a) A \subsetneq P_{pr}; \quad b) P_r \subsetneq P_{cr}; \quad c) P_{pr} \subsetneq P_r,$$

т.е. все вложения в (10) — собственные.

И пока остается самый главный вопрос: как связаны классы рекурсивных и вычислимых функций. Обсуждать этот вопрос будем на следующей лекции.